BÁO CÁO THỰC HÀNH KIẾN TRÚC MÁY TÍNH

LAB 1: ARITHMETIC INSTRUCTIONS

Thông tin sinh viên: - **Họ và tên:** Đoàn Duy Tùng - **MSSV:** *2033364* - **Lớp:** *L01* - **Ngày nộp:** 22/10/2026 - **Kho lưu trữ mã nguồn trực tuyến (GitHub Repository):** https://github.com/DuytungDeveloper/CA-Lab01-L01-2033364 - **Danh mục mã nguồn Assembly MIPS:** `ex1.asm`, `ex2.asm`, `ex3.asm`, `ex4.asm`, `ex5.asm`

> **Ghi chú về phương pháp trình bày mã nguồn:**
> Tuân thủ đúng định hướng của đề cương môn học: *"Ở những bài yêu cầu code assembly MIPS, giải thích cách giải qua code. (Không cần thiết phải copy+paste toàn bộ code vào báo cáo, SV có thể chọn lọc phần quan trọng để minh họa hoặc upload lên các repo online)"*.
> Do hình thức nộp bài là tài liệu PDF độc lập, báo cáo này chọn lọc trích dẫn và phân tích trực tiếp các khối lệnh MIPS then chốt (phân bổ bộ nhớ, kỹ thuật trải mã, cấu trúc điều khiển rẽ nhánh, các phép toán số học và cơ chế bẫy ngoại lệ phần cứng). Toàn bộ mã nguồn hoàn chỉnh với đầy đủ chú thích kiến trúc và kịch bản kiểm thử trên MARS 4.5 đều được đồng bộ tại repository GitHub: https://github.com/DuytungDeveloper/CA-Lab01-L01-2033364.

Bài 1 (Exercise 1)

Yêu cầu: Viết chương trình MIPS nhập tên từ bàn phím và in ra `Hello <name>!`.

1. Giải thích giải thuật qua các đoạn mã MIPS then chốt:

- **Đọc chuỗi an toàn:** Sử dụng `syscall 8` (`read_string`), nạp `$a1 = 100` để giới hạn bộ nhớ đệm tiếp nhận tối đa 99 ký tự và kết thúc bằng `\0`, chống tràn bộ nhớ (Buffer Overflow).
- **Bắt ngoại lệ chuỗi rỗng:** Kiểm tra byte đầu tiên `buffer[0]`. Nếu người dùng chỉ nhấn Enter mà không nhập gì (`buffer[0] == '\n'` mã 10 hoặc `\0`), chương trình rẽ nhánh đến `error_empty` và dừng an toàn:

```
# Kiểm tra chuỗi rỗng (Người dùng chỉ ấn Enter)
la   $t0, buffer            # $t0 trỏ vào ký tự đầu của chuỗi
lb   $t1, 0($t0)            # Đọc byte đầu tiên
beq  $t1, 10, error_empty   # Nếu là '\n' -> Báo lỗi rỗng
beqz $t1, error_empty       # Nếu là '\0' -> Báo lỗi rỗng
```

- **Xử lý xóa `\n` và chèn `!`:** Lệnh `syscall 8` trong MARS tự động lưu cả ký tự xuống dòng `\n` khi người dùng nhấn Enter. Chương trình duyệt chuỗi bằng con trỏ `$t0`, thay thế ký tự `\n` (ASCII 10) thành dấu chấm than `!` (ASCII 33) và chèn ký tự kết thúc `\0` ngay sau đó:

```
remove_nl:
    lb   $t1, 0($t0)
    beqz $t1, print_hello       # Gặp '\0' thì dừng duyệt
    beq  $t1, 10, replace_nl    # Gặp '\n' thì nhảy đến thay thế
    addi $t0, $t0, 1            # Sang byte tiếp theo
    j    remove_nl

replace_nl:
    li   $t2, 33                # Mã ASCII của '!' là 33
    sb   $t2, 0($t0)            # Ghi đè '!' vào vị trí '\n'
    sb   $zero, 1($t0)          # Ghi '\0' liền sau dấu '!'
```

2. Kết quả chạy thử trên MARS 4.5:

- **Trường hợp hợp lệ:**
  - Input: `John`
  - Output: `Hello, John!`
- **Trường hợp ngoại lệ (ấn Enter ngay):**
  - Input: `\n`
  - Output: `Error: Name cannot be empty!`

(Mã nguồn hoàn chỉnh có chú thích chi tiết được lưu trữ tại: GitHub - ex1.asm)

Bài 2 (Exercise 2)

Yêu cầu: Cho người dùng nhập vào mảng 5 số nguyên. Sau đó nhập index (từ 0 đến 4) và in ra giá trị phần tử tại index đó.

1. Cơ sở lý thuyết Kiến trúc máy tính: Tại sao Offset lại là 0, 4, 8, 12, 16,...?

- **Nguyên lý Định địa chỉ theo từng Byte (Byte Addressing):** Trong kiến trúc máy tính MIPS, bộ nhớ RAM được tổ chức và quản lý theo từng Byte (1 Byte = 8 bits). Mỗi ô nhớ vật lý trên RAM lưu đúng 1 Byte và được gán 1 địa chỉ riêng biệt.
- **Kích thước kiểu dữ liệu Từ nhớ (Word - 32 bit):** Một số nguyên 32-bit (kiểu `int` trong C/C# hay chỉ thị `.word` trong MIPS) chiếm $32 \text{ bits} = 4 \text{ Bytes}$ $(32/8 = 4)$. Do đó, để lưu trữ trọn vẹn một phần tử số nguyên, CPU bắt buộc phải cấp phát một khối gồm 4 Bytes liên tiếp nhau trong RAM.
- **Quy tắc tính độ lệch byte (Offset):**
  - Mảng 5 phần tử bắt đầu tại địa chỉ cơ sở Base (lưu trong `$s0`).
  - Do mỗi phần tử 32-bit chiếm 4 bytes, các phần tử lần lượt nằm tại các địa chỉ:
    - array[0]: $\text{Base} + 0$ $(\text{Offset} = 0)$
    - array[1]: $\text{Base} + 4$ $(\text{Offset} = 4)$
    - array[2]: $\text{Base} + 8$ $(\text{Offset} = 8)$
    - array[3]: $\text{Base} + 12$ $(\text{Offset} = 12)$
    - array[4]: $\text{Base} + 16$ $(\text{Offset} = 16)$

2. Giải thích các lệnh then chốt trong mã nguồn:

- **Căn chỉnh ranh giới từ nhớ (Word Alignment - `.align 2`):** Trong MIPS 32-bit, mọi ô nhớ chứa dữ liệu kiểu Word bắt buộc phải có địa chỉ chia hết cho 4. Do các chuỗi `.asciiz` ở phía trước có độ dài byte lẻ, nếu không có chỉ thị `.align 2`, mảng `array` có thể bị đặt vào địa chỉ lẻ, gây ra ngoại lệ ngắt phần cứng `store address not aligned on word boundary`. Chỉ thị `.align 2` ép vùng nhớ bắt đầu tại bội số của $2^2 = 4$ bytes.
- **Cơ chế phòng thủ 2 tầng cho dữ liệu đầu vào và chỉ số mảng:**
  - *Tầng 1 (Bảo vệ phần cứng / MARS Kernel Runtime):* Nếu người dùng nhập giá trị phần tử hoặc chỉ số `index` vượt quá dải số nguyên 32-bit có dấu $[-2^{31}, 2^{31}-1]$ (ví dụ: `5000000000`, `9999999999999999`) hoặc nhập ký tự chữ: `syscall 5` (`read_int` của MARS, sử dụng `Integer.parseInt`) sẽ tự động kích hoạt ngoại lệ ngắt `Runtime exception: invalid integer input (syscall 5)` và dừng chương trình an toàn. Giá trị không hợp lệ không được ghi vào thanh ghi `$v0` hay vùng nhớ RAM. Đồng thời, do các phần tử mảng chỉ được nạp (`lw`) và lưu (`sw`), không thực hiện phép tính nhân/chia dồn như Bài 3 nên không phát sinh hiện tượng tràn số trong quá trình tính toán.
  - *Tầng 2 (Bảo vệ luận lý phần mềm):* Đối với các số nguyên 32-bit hợp lệ do người dùng nhập vào, chương trình chủ động kiểm tra ranh giới hợp lệ của `index` trong đoạn $[0, 4]$ bằng các lệnh rẽ nhánh:

```
# Kiểm tra ranh giới: index < 0 hoặc index > 4
bltz $v0, error_index         # Nếu index < 0 -> Báo lỗi
li   $t9, 4
bgt  $v0, $t9, error_index   # Nếu index > 4 -> Báo lỗi
```

- **Nhập dữ liệu tuần tự tuân thủ tiêu chí Lab 1 (Kỹ thuật Trải mã tuần tự - Loop Unrolling):** Tuân thủ yêu cầu đề cương tuần 1 (chỉ sử dụng lệnh tuần tự, số học và truyền dữ liệu, không dùng vòng lặp), chương trình nạp trực tiếp giá trị vào từng ô nhớ RAM thông qua các độ lệch offset cố định:

```
# Nhập phần tử 0: Offset = 0 * 4 = 0 bytes
li   $v0, 4; la $a0, prompt0; syscall
li   $v0, 5; syscall
sw   $v0, 0($s0)            # Ghi vào ô nhớ Base + 0

# Nhập phần tử 1: Offset = 1 * 4 = 4 bytes
li   $v0, 4; la $a0, prompt1; syscall
li   $v0, 5; syscall
sw   $v0, 4($s0)            # Ghi vào ô nhớ Base + 4

# ... Tương tự với phần tử 2 (offset 8), phần tử 3 (offset 12) ...

# Nhập phần tử 4: Offset = 4 * 4 = 16 bytes
li   $v0, 4; la $a0, prompt4; syscall
li   $v0, 5; syscall
sw   $v0, 16($s0)           # Ghi vào ô nhớ Base + 16
```

- **Truy xuất phần tử tức thì trong thời gian $O(1)$:** Sau khi người dùng nhập `index`, CPU tính địa chỉ trong 1 lệnh dịch bit và 1 lệnh cộng:

```
sll  $t0, $v0, 2        # $t0 = index * 4 (độ lệch byte trong RAM)
add  $t1, $s0, $t0      # $t1 = $s0 + $t0 (địa chỉ tuyệt đối của array[index])
lw   $a0, 0($t1)        # Nạp giá trị 4 bytes từ RAM vào $a0
li   $v0, 1             # In số nguyên ra màn hình
syscall
```

3. Kết quả chạy thử trên MARS 4.5:

- **Trường hợp hợp lệ:**
  - Nhập mảng: `10, 20, 30, 40, 50`
  - Nhập index: `3` → Output: `40`
- **Trường hợp ngoại lệ (vượt cận trên):**
  - Nhập index: `5` → Output: `Error: Index out of bounds (must be between 0 and 4)!`
- **Trường hợp ngoại lệ (chỉ số âm):**
  - Nhập index: `-1` → Output: `Error: Index out of bounds (must be between 0 and 4)!`
- **Trường hợp ngoại lệ giá trị vượt ngưỡng 32-bit (Tầng 1):**
  - Nhập phần tử hoặc index: `5000000000` → Output: `Runtime exception: invalid integer input (syscall 5)` (chương trình dừng an toàn, không làm hỏng bộ nhớ).

(Mã nguồn hoàn chỉnh có chú thích chi tiết được lưu trữ tại: GitHub - ex2.asm)

Bài 3 (Exercise 3)

Yêu cầu: Nhập vào 4 số nguyên a, b, c, d. Tính giá trị biểu thức:

$$F = \frac{(a+10) \times (b-d) \times (c-2 \times a)}{a+b+c}$$

In ra cả phần thương (quotient) và phần dư (remainder).

1. Giải thích giải thuật qua các đoạn mã MIPS then chốt:

- **Tính các thừa số của Tử số:**
  - Thừa số $(a+10)$: dùng lệnh `addi $t0, $s0, 10`.
  - Thừa số $(b-d)$: dùng lệnh `sub $t1, $s1, $s3`.
  - Thừa số $(c-2a)$: Tính $2a$ bằng phép dịch trái 1 bit `sll $t2, $s0, 1`, sau đó trừ `sub $t2, $s2, $t2`.
- **Nhân tích lũy Tử số kết hợp kiểm tra tràn số có dấu 32-bit:** Thay vì dùng lệnh `mul` (âm thầm bỏ rơi các bit tràn và lật bit dấu gây Silent Failure), chương trình sử dụng lệnh nhân 64-bit `mult` và kiểm tra thanh ghi `HI`: nếu `HI` khác phần mở rộng dấu của `LO` (`sra $t6, $t3, 31`), hệ thống lập tức bắt ngoại lệ và dừng an toàn:

```
# Lần nhân 1: (a + 10) * (b - d)
mult $t0, $t1               # Nhân 64-bit vào cặp HI:LO
mflo $t3                    # LO = 32-bit thấp
mfhi $t5                    # HI = 32-bit cao
sra  $t6, $t3, 31           # Phủ bit dấu của LO ra toàn bộ $t6
bne  $t5, $t6, error_overflow   # Nếu HI != sign(LO) -> Báo lỗi tràn số ngay!

# Lần nhân 2: Nhân tiếp với thừa số 3 (c - 2a)
mult $t3, $t2
mflo $t3
mfhi $t5
sra  $t6, $t3, 31
bne  $t5, $t6, error_overflow   # Nếu tràn số 32-bit -> Báo lỗi!
```

- **Tính Mẫu số và Bắt ngoại lệ chia cho 0:**

```
# Mẫu số = a + b + c
add  $t4, $s0, $s1
add  $t4, $t4, $s2

# Kiểm tra ngoại lệ chia cho 0 (Divide by Zero)
beqz $t4, error_div_zero      # Nếu mẫu số == 0 -> Rẽ nhánh báo lỗi
```

- **Thực thi phép chia trong mạch ALU phần cứng MIPS:** Lệnh `div $t3, $t4` trong MIPS đồng thời tính cả thương số và số dư trong mạch chia phần cứng:
  - Thương số tự động lưu vào thanh ghi phần cứng `LO`.
  - Số dư tự động lưu vào thanh ghi phần cứng `HI`.

```
div  $t3, $t4            # Chia Tử số ($t3) cho Mẫu số ($t4)
mflo $s4                # Lấy Thương từ LO vào $s4
mfhi $s5                # Lấy Số dư từ HI vào $s5
```

2. Phân tích hậu quả nếu không xử lý các ngoại lệ (Góc độ Kiến trúc phần cứng):

- **Trường hợp Mẫu số bằng 0 ($a+b+c=0$):**
  - Trong vi xử lý MIPS thực tế và trình mô phỏng MARS, mạch chia ALU khi gặp số chia bằng 0 không tự động phát sinh ngắt phần cứng (Hardware Trap).
  - Phần cứng MIPS để mặc định giá trị trong cặp thanh ghi `LO` và `HI` (trên MARS được gán `LO = 0, HI = 0`).
  - *Hậu quả:* Chương trình tiếp tục thực thi các lệnh `mflo`, `mfhi` và in ra kết quả $F = 0, \text{remainder } 0$. Đây là hiện tượng **Silent Failure** (lỗi logic ngầm): chương trình không dừng nhưng cho ra kết quả sai lệch so với bản chất toán học (phép chia cho 0 không xác định).
  - Do đó, cần chủ động kiểm tra mẫu số bằng lệnh `beqz $t4, error_div_zero` trước khi gọi lệnh `div`.
- **Trường hợp Tràn số nguyên có dấu 32-bit (Signed Overflow) ở Tử số:**
  - Miền giá trị số nguyên 32-bit có dấu là $[-2^{31}, 2^{31}-1]$ (khoảng $[-2.147 \times 10^9, +2.147 \times 10^9]$).
  - Tử số là tích của 3 thừa số. Khi giá trị vượt quá dải trên (ví dụ $a = 20000, b = 30000, c = 50000, d = 0 \Longrightarrow \text{Tử số} \approx 6 \times 10^{12}$), lệnh giả `mul` chỉ lưu 32 bit thấp, dẫn đến lật bit dấu và biến kết quả thành số âm sai lệch.

- Sử dụng lệnh nhân phần cứng `mult` kết hợp so sánh `HI` với phần mở rộng dấu `sra 31` của `LO` bằng lệnh `bne` giúp phát hiện và bắt ngoại lệ tràn số chính xác.

3. Bộ trường hợp kiểm thử (Testcases) chi tiết:

- **Testcase 1 (Số dương chuẩn):**
  - Input: $a = 3, b = 4, c = 5, d = 6$
  - Tử số: $(3 + 10) \times (4 - 6) \times (5 - 6) = 13 \times (-2) \times (-1) = 26$
  - Mẫu số: $3 + 4 + 5 = 12$
  - Phép chia: $26/12 = 2$ dư 2
  - Output: `F = 2, remainder 2`
- **Testcase 2 (Số âm, kết quả âm):**
  - Input: $a = 1, b = 2, c = 3, d = 5$
  - Tử số: $(1 + 10) \times (2 - 5) \times (3 - 2) = 11 \times (-3) \times 1 = -33$
  - Mẫu số: $1 + 2 + 3 = 6$
  - Phép chia trong MIPS: $-33/6 = -5$ dư -3 (theo chuẩn MIPS: số dư cùng dấu với số bị chia: $-33 = 6 \times (-5) + (-3)$)
  - Output: `F = -5, remainder -3`
- **Testcase 3 (Ngoại lệ Mẫu số bằng 0):**
  - Input: $a = 2, b = -2, c = 0, d = 5$ (Mẫu số $= 2 + (-2) + 0 = 0$)
  - Output: `Error: Denominator (a + b + c) equals 0! Division by zero is not allowed.`
- **Testcase 4 (Ngoại lệ Mẫu số bằng 0 do triệt tiêu đối xứng):**
  - Input: $a = 10, b = -5, c = -5, d = 1$ (Mẫu số $= 10 + (-5) + (-5) = 0$)
  - Output: `Error: Denominator (a + b + c) equals 0! Division by zero is not allowed.`
- **Testcase 5 (Thương bằng 0, chỉ có số dư):**
  - Input: $a = 0, b = 1, c = 10, d = 0$
  - Tử số: $(0 + 10) \times (1 - 0) \times (10 - 0) = 100$
  - Mẫu số: $0 + 1 + 10 = 11$
  - Phép chia: $100/11 = 9$ dư 1
  - Output: `F = 9, remainder 1`
- **Testcase 6 (Ngoại lệ Tràn số 32-bit ở Tử số):**
  - Input: $a = 20000, b = 30000, c = 50000, d = 0$
  - Tử số: $20010 \times 30000 \times 10000 = 6.003 \times 10^{12} > 2.147 \times 10^{9}$
  - Output: `Error: Arithmetic overflow! Value exceeds 32-bit system capability.`

(Mã nguồn hoàn chỉnh có chú thích chi tiết được lưu trữ tại: GitHub - ex3.asm)

Bài 4 (Exercise 4)

Yêu cầu đề bài: Nhập số nguyên dương $N < 16$, in dạng biểu diễn nhị phân 4-bit của số đó.

1. Giải thích giải thuật qua các đoạn mã MIPS then chốt (Bản chuẩn 4-bit):

- **Bắt ngoại lệ ngoài khoảng 4-bit ($0 \le N \le 15$):** Nếu người dùng nhập $N < 0$ hoặc $N \ge 16$, chuỗi nhị phân cần nhiều hơn 4 bit để biểu diễn. Nếu vẫn ép in 4 bit, chương trình sẽ cắt cụt các bit cao dẫn đến sai lệch ngữ nghĩa nghiêm trọng. Đoạn mã chặn đứng lỗi ngay từ đầu:

```
bltz $s0, error_range         # Nếu N < 0 -> Báo lỗi ngoài khoảng
li   $t9, 15
bgt  $s0, $t9, error_range    # Nếu N > 15 -> Báo lỗi ngoài khoảng
```

- **Trích xuất từng bit bằng phép dịch phải logic (`srl`) và mặt nạ bit (`andi`):** Số nguyên 4-bit được biểu diễn dưới dạng $b_3b_2b_1b_0$. Để in ra chuỗi nhị phân từ trái sang phải:
  - **Bit 3 (MSB, trọng số $2^3 = 8$):** Dịch phải 3 bit (`srl $t0, $s0, 3`), đưa bit 3 về vị trí bit 0, sau đó giữ lại bit tận cùng bằng `andi $t0, $t0, 1` rồi in qua Syscall 1:

```
srl  $t0, $s0, 3            # Đưa bit 3 về vị trí bit 0
andi $t0, $t0, 1            # Giữ lại bit tận cùng
li   $v0, 1
move $a0, $t0
syscall                     # In ra ký số bit 3
```

- **Bit 2 (trọng số $2^2 = 4$):** `srl $t0, $s0, 2` → `andi $t0, $t0, 1` → in ra.
- **Bit 1 (trọng số $2^1 = 2$):** `srl $t0, $s0, 1` → `andi $t0, $t0, 1` → in ra.
- **Bit 0 (LSB, trọng số $2^0 = 1$):** Không cần dịch, lấy trực tiếp `andi $t0, $s0, 1` → in ra.

2. Kết quả chạy thử trên MARS 4.5:

- **Trường hợp hợp lệ:**
  - Nhập $N = 10$ → Output: `Its binary form is: 1010`
  - Nhập $N = 15$ → Output: `Its binary form is: 1111`
  - Nhập $N = 7$ → Output: `Its binary form is: 0111`
- **Trường hợp ngoại lệ (nhập ngoài khoảng):**
  - Nhập $N = 16$ → Output: `Error: Input must be a positive integer less than 16 (0 to 15)!`
  - Nhập $N = -3$ → Output: `Error: Input must be a positive integer less than 16 (0 to 15)!`

(Mã nguồn hoàn chỉnh có chú thích chi tiết được lưu trữ tại: GitHub - ex4.asm)

Bài 5 (Exercise 5)

Yêu cầu: Khai báo một mảng 10 số nguyên cho trước, in các phần tử theo thứ tự đảo ngược.

1. Giải thích giải thuật qua các đoạn mã MIPS then chốt:

- Mảng khởi tạo trong `.data` : `array: .word 1, 2, 5, 8, 12, 44, 3, 9, 0, 10` (10 từ nhớ $= 40$ bytes liên tục trong RAM).
- Phần tử cuối cùng `array[9]` nằm tại offset $9 \times 4 = 36$ bytes. Phần tử đầu tiên `array[0]` nằm tại offset 0 bytes.
- **Kỹ thuật Trải mã tuần tự (Loop Unrolling - Phương án triển khai chính):**

- Mảng 10 số nguyên 32-bit chiếm 40 bytes liên tục trong bộ nhớ RAM, bắt đầu từ địa chỉ cơ sở `$s0`.
- Để in đảo ngược, chương trình đọc tuần tự từ phần tử cuối cùng `array[9]` (nằm tại offset $9 \times 4 = 36$ bytes) lùi dần về phần tử đầu tiên `array[0]` (offset 0 bytes).
- Riêng phần tử cuối cùng in ra (offset 0) không in kèm dấu phẩy `", "`:

```
# Đọc tuần tự từ phần tử cuối lùi dần về đầu:
lw   $a0, 36($s0)       # array[9] = 10
li   $v0, 1             # In số nguyên
syscall
li   $v0, 4             # In chuỗi ", "
la   $a0, sep
syscall

# ... Lặp lại tuần tự cho các offset 32, 28, 24, 20, 16, 12, 8, 4 ...

lw   $a0, 0($s0)        # array[0] = 1 (phần tử cuối, không in dấu phẩy)
li   $v0, 1
syscall
```

- **Đánh giá hiệu năng và tính an toàn:**
  - *Hiệu năng:* Thực thi tuần tự các lệnh nạp/in trực tiếp không dùng rẽ nhánh, loại bỏ chi phí trễ nhánh (Branch Penalty).
  - *Tính an toàn ngoại lệ và trường hợp giá trị > 32-bit:*
    - **Thiết kế tĩnh hiện tại:** Mảng được khởi tạo cố định trong vùng dữ liệu `.data` (`User Input = None`), các chỉ số offset được lập trình cố định từ 36 về 0. Do đó chương trình đảm bảo an toàn truy xuất bộ nhớ, không xảy ra ngoại lệ ngoài ranh giới khi thực thi. Nếu khai báo giá trị vượt quá dải số nguyên 32-bit có dấu trong `.word` (ví dụ: `5000000000`), Trình dịch MARS sẽ báo lỗi ngay tại pha hợp dịch (`is not a valid 32-bit integer`), mã nguồn không thể hợp dịch thành mã máy để thực thi.
    - **Kịch bản mở rộng (nhập từ bàn phím):** Nếu mở rộng cho người dùng nhập mảng qua `syscall 5`, các giá trị đầu vào > 32-bit và ký tự bất hợp lệ được tầng Kernel tự động chặn lại tương tự Bài 2; đồng thời do chỉ nạp và in nên không phát sinh hiện tượng tràn số trong tính toán.

2. Kết quả chạy thử trên MARS 4.5:

- Kết quả in ra màn hình: `10, 0, 9, 3, 44, 12, 8, 5, 2, 1`

(Mã nguồn hoàn chỉnh có chú thích chi tiết được lưu trữ tại: GitHub - ex5.asm)

Bài 6 (Exercise 6)

Đề bài: Hệ thống màn hình hiển thị: - Mỗi màu biểu diễn bởi 8-bit, mỗi pixel gồm 3 màu (RGB). - Độ phân giải: 1280×1024.

Bài giải chi tiết và đáp số:

a. Xác định dung lượng tối thiểu của mỗi khung hình (frame buffer):

- Tổng số lượng pixel trong một khung hình:

$$\text{Số pixels} = 1280 \times 1024 = 1,310,720 \text{ pixels}$$

- Kích thước biểu diễn của mỗi pixel:

$$\text{Kích thước/pixel} = 3 \times 8 \text{ bits} = 24 \text{ bits} = 3 \text{ Bytes}$$

- Dung lượng tối thiểu của khung hình:
  - Tính theo đơn vị bit:
$$\text{Dung lượng (bits)} = 1,310,720 \times 24 = 31,457,280 \text{ bits}$$
  - Tính theo đơn vị Byte và MiB:
$$\text{Dung lượng (Bytes)} = \frac{31,457,280}{8} = 3,932,160 \text{ Bytes}$$
$$\text{Dung lượng (MiB)} = \frac{3,932,160}{1024 \times 1024} = 3.75 \text{ MiB}$$

> **Đáp số câu a:** - $31,457,280$ bits - $3,932,160$ Bytes **(**3.75 MiB**)**

b. Thời gian truyền khung hình qua mạng tốc độ 100 Mbit/s:

- Tốc độ đường truyền mạng:
$$R = 100 \text{ Mbit/s} = 100 \times 10^6 \text{ bits/s}$$
- Thời gian truyền một khung hình (T):
$$T = \frac{\text{Dung lượng khung hình (bits)}}{R} = \frac{31,457,280 \text{ bits}}{100 \times 10^6 \text{ bits/s}} = 0.3145728 \text{ s} \approx 314.57 \text{ ms}$$

> **Đáp số câu b:** - 0.3145728 giây **(khoảng** 314.57 ms**)**

Bài 7 (Exercise 7)

Đề bài: Ba bộ xử lý thực thi cùng một tập lệnh: - Bộ xử lý P_1: Clock Rate = 3.0 GHz, CPI = 1.5 - Bộ xử lý P_2: Clock Rate = 2.5 GHz, CPI = 1.0 - Bộ xử lý P_3: Clock Rate = 4.0 GHz, CPI = 2.2

Bài giải chi tiết và đáp số:

a. Bộ xử lý có hiệu năng cao nhất theo IPS (Instructions Per Second):

- Công thức tính số lệnh thực thi trong 1 giây (IPS):
$$\text{IPS} = \frac{\text{Clock Rate}}{\text{CPI}}$$
- Tính toán cho từng bộ xử lý:
  - Với P_1:
$$\text{IPS}(P_1) = \frac{3.0 \times 10^9 \text{ cycles/s}}{1.5 \text{ cycles/instruction}} = 2.0 \times 10^9 \text{ instructions/s} = 2.0 \text{ GIPS}$$
  - Với P_2:
$$\text{IPS}(P_2) = \frac{2.5 \times 10^9 \text{ cycles/s}}{1.0 \text{ cycles/instruction}} = 2.5 \times 10^9 \text{ instructions/s} = 2.5 \text{ GIPS}$$
  - Với P_3:
$$\text{IPS}(P_3) = \frac{4.0 \times 10^9 \text{ cycles/s}}{2.2 \text{ cycles/instruction}} \approx 1.818 \times 10^9 \text{ instructions/s} \approx 1.818 \text{ GIPS}$$

> **Đáp số câu a:** - **Bộ xử lý** P_2 **có hiệu năng cao nhất** vì đạt tốc độ thực thi lớn nhất (2.5 GIPS).

b. Số chu kỳ xung nhịp và số lệnh thực thi trong 10 giây ($T = 10$s):

- Công thức:
  - Số chu kỳ (Cycles) $= T \times$ Clock Rate

- Số lệnh (Instructions) $= \text{IPS} \times T = \frac{\text{Cycles}}{\text{CPI}}$

- Tính toán cho từng bộ xử lý:
  - **Bộ xử lý P_1:**
    $$\text{Cycles} = 10 \times (3.0 \times 10^9) = 3.0 \times 10^{10} \text{ chu kỳ}$$
    $$\text{Instructions} = 10 \times (2.0 \times 10^9) = 2.0 \times 10^{10} \text{ lệnh}$$
  - **Bộ xử lý P_2:**
    $$\text{Cycles} = 10 \times (2.5 \times 10^9) = 2.5 \times 10^{10} \text{ chu kỳ}$$
    $$\text{Instructions} = 10 \times (2.5 \times 10^9) = 2.5 \times 10^{10} \text{ lệnh}$$
  - **Bộ xử lý P_3:**
    $$\text{Cycles} = 10 \times (4.0 \times 10^9) = 4.0 \times 10^{10} \text{ chu kỳ}$$
    $$\text{Instructions} = 10 \times \left(\frac{4.0 \times 10^9}{2.2}\right) \approx 1.818 \times 10^{10} \text{ lệnh}$$

> **Đáp số câu b:** - P_1**:** 3×10^{10} chu kỳ, 2×10^{10} lệnh. - P_2**:** 2.5×10^{10} chu kỳ, 2.5×10^{10} lệnh. - P_3**:** 4×10^{10} chu kỳ, 1.818×10^{10} lệnh.

c. Thời gian thực thi giảm 30%, CPI tăng 20%. Tính tần số xung nhịp mới:

- Gọi các thông số ban đầu là T, CPI, f.
- Thời gian mới: $T' = T \times (1 - 0.3) = 0.7 \times T$.
- CPI mới: $\text{CPI}' = \text{CPI} \times (1 + 0.2) = 1.2 \times \text{CPI}$.
- Số lượng lệnh I của chương trình không đổi.
- Ta có mối quan hệ:
$$T = \frac{I \times \text{CPI}}{f} \Longrightarrow f = \frac{I \times \text{CPI}}{T}$$
$$f' = \frac{I \times \text{CPI}'}{T'} = \frac{I \times (1.2 \times \text{CPI})}{0.7 \times T} = \frac{1.2}{0.7} \times f = \frac{12}{7} \times f \approx 1.7143 \times f$$
- Tính tần số mới cho từng bộ xử lý:
  - Với P_1:
    $$f_1' = \frac{12}{7} \times 3.0 \text{ GHz} \approx 5.143 \text{ GHz}$$
  - Với P_2:
    $$f_2' = \frac{12}{7} \times 2.5 \text{ GHz} \approx 4.286 \text{ GHz}$$
  - Với P_3:
    $$f_3' = \frac{12}{7} \times 4.0 \text{ GHz} \approx 6.857 \text{ GHz}$$

> **Đáp số câu c:** - Tần số xung nhịp mới cần tăng lên gấp $\frac{12}{7} \approx 1.7143$ **lần** tần số cũ. - P_1'**:** ≈ 5.143 GHz - P_2'**:** ≈ 4.286 GHz - P_3'**:** ≈ 6.857 GHz

---

Bảng tổng kết cơ chế xử lý ngoại lệ trong toàn bộ Lab 1

Bài tập	Loại ngoại lệ có thể gặp	Hậu quả nếu không bắt	Giải pháp trong mã MIPS	Tương đương trong C# / C
Bài 1	Chuỗi rỗng (nhấn Enter ngay)	In lời chào rỗng: `Hello, !`	Kiểm tra byte đầu: `beq $t1, 10, error_empty`	`string.IsNullOrWhiteSpace()`
Bài 1	Nhập chuỗi quá 100 ký tự	Tràn bộ đệm (Buffer Overflow)	Đặt `$a1 = 100` khi gọi `syscall 8`	`fgets(buf, 100, stdin)`
Bài 2 & 3	Nhập số > 32-bit hoặc chuỗi chữ	Dữ liệu vượt dung lượng từ nhớ 32-bit	Tầng 1: `syscall 5` Runtime Trap (`invalid integer input`)	`FormatException` / `OverflowException`
Bài 2	`index < 0` hoặc `index > 4`	Đọc rác ngoài mảng / Crash bộ nhớ	Tầng 2: Kiểm tra `bltz $v0` và `bgt $v0, 4`	`IndexOutOfRangeException`
Bài 2	Địa chỉ không chia hết cho 4	Lỗi phần cứng Alignment Exception	Chỉ thị `.align 2` trước mảng	Căn lề địa chỉ 4-byte
Bài 3	Mẫu số $a + b + c = 0$	Lỗi chia cho 0, Silent Failure (LO=0)	Kiểm tra `beqz $t4, error_div_zero`	`DivideByZeroException`
Bài 3	Tràn số Tử số 32-bit	Kết quả sai lệch hoàn toàn, lật bit dấu	`mult` kết hợp `sra $t6, $t3, 31` và `bne`	`OverflowException`
Bài 4	$N < 0$ hoặc $N \geq 16$	Sai lệch ngữ nghĩa biểu diễn nhị phân	Kiểm tra `bltz $s0` và `bgt $s0, 15`	`ArgumentOutOfRangeException`
Bài 5	Giá trị > 32-bit / Vượt ranh giới	Tràn số / Crash bộ nhớ	Mảng tĩnh, MARS Assembler chặn >32-bit; offset chốt cứng 36..0	Compile-time bounds checking